

Developer Evaluation Questions

Solutions and Feedback

Both questions are suitable for an initial coding screen. Below are the solution approaches, Java implementations, and brief feedback on their usefulness in interviews.

1 High Level Solution Details

Are They Pangrams. Scan each string and track which of the 26 English letters have appeared. Repeated letters do not affect the result. Once every letter is present, the string is a pangram. Append 1 for a pangram and 0 otherwise, then return the combined result in input order. A bit mask tracks the letters using constant space.

REST API Most Active Authors. Request the first page and read the total number of pages. Process every page, checking each user's `submission_count` against the threshold. Add the username only when the count is strictly greater than the threshold. Preserve response order and return the collected usernames after all pages have been processed. Each page is fetched once, and each user is checked once.

2 Are They Pangrams

Track the English letters present in each string using a 26-bit mask. Return 1 when all letters are present and 0 otherwise, preserving input order. The implementation handles uppercase letters and ignores non-letter characters.

Efficiency. Linear time in the total input size and constant additional space per string, excluding the output.

2.1 Java implementation

```
package interview;

import java.util.List;
import java.util.Objects;

public final class Pangrams {
    private static final int ALL_LETTERS = (1 << 26) - 1;
    private Pangrams() {}

    /** ASCII English letters, case insensitive; other characters are ignored. */
    public static boolean isPangram(String text) {
        Objects.requireNonNull(text, "text");
        int seen = 0;
        for (int i = 0; i < text.length(); i++) {
            char c = text.charAt(i);
            if (c >= 'A' && c <= 'Z') c = (char) (c + ('a' - 'A'));
            if (c >= 'a' && c <= 'z') {
                seen |= 1 << (c - 'a');
                if (seen == ALL_LETTERS) return true;
            }
        }
        return false;
    }
}
```

```

    }

    public static String areTheyPangrams(List<String> strings) {
        Objects.requireNonNull(strings, "strings");
        StringBuilder result = new StringBuilder(strings.size());
        for (String text : strings) result.append(isPangram(text) ? '1' : '0');
        return result.toString();
    }
}

```

3 REST API Most Active Authors

Fetch the first API page, read `total_pages`, and process all remaining pages. Return usernames whose `submission_count` is strictly greater than the threshold, in the order they appear in the responses.

Efficiency. Each page is requested once and each user is checked once. Memory holds one page at a time and the result list.

The response is mapped to nested `AuthorPage` and `Author` models. Field names match the API JSON, so annotations are unnecessary. The models include all fields described in the question, while validation checks the fields required by the solution. Additional unknown fields are ignored. The GET request needs only a page query parameter and no request body model.

3.1 Java implementation

```

package interview;
import com.fasterxml.jackson.databind.DeserializationFeature;
import com.fasterxml.jackson.databind.ObjectMapper;
import java.io.IOException;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.time.Duration;
import java.util.ArrayList;
import java.util.List;
import java.util.Objects;
public final class ActiveAuthors {
    private static final URI ENDPOINT = URI.create(
        "https://jsonmock.hackerrank.com/api/article_users");
    private final HttpClient client;
    private final URI endpoint;
    private final ObjectMapper mapper = new ObjectMapper()
        .disable(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES)
        .disable(DeserializationFeature.ACCEPT_FLOAT_AS_INT);
    public ActiveAuthors() {
        this(HttpClient.newBuilder()
            .connectTimeout(Duration.ofSeconds(10))
            .build(), ENDPOINT);
    }
    /** An alternate endpoint allows deterministic tests against a local server. */
    public ActiveAuthors(HttpClient client, URI endpoint) {
        this.client = Objects.requireNonNull(client);
        this.endpoint = Objects.requireNonNull(endpoint);
        if (endpoint.getRawQuery() != null || endpoint.getFragment() != null)
            throw new IllegalArgumentException("Endpoint must have no query or fragment");
    }
}

```

```

    }
    public List<String> getUsernames(int threshold)
        throws IOException, InterruptedException {
        List<String> result = new ArrayList<>();
        AuthorPage first = fetchPage(1);
        int totalPages = first.total_pages;
        appendMatches(first, threshold, result);
        for (int page = 2; page <= totalPages; page++) {
            AuthorPage response = fetchPage(page);
            if (response.total_pages != totalPages)
                throw new IOException("Pagination changed during traversal");
            appendMatches(response, threshold, result);
        }
        return result;
    }
    private AuthorPage fetchPage(int page) throws IOException, InterruptedException {
        HttpRequest request = HttpRequest.newBuilder(URI.create(endpoint + "?page=" + page))
            .timeout(Duration.ofSeconds(15)).header("Accept", "application/json").GET().build();
        HttpResponse<String> response = client.send(
            request, HttpResponse.BodyHandlers.ofString());
        if (response.statusCode() != 200)
            throw new IOException("HTTP " + response.statusCode());
        AuthorPage parsed = mapper.readValue(
            response.body(), AuthorPage.class);
        validatePage(parsed, page);
        return parsed;
    }

    private static void validatePage(AuthorPage response, int requestedPage)
        throws IOException {
        if (response == null || response.page == null
            || response.total_pages == null || response.data == null)
            throw new IOException("Missing page fields");
        int totalPages = response.total_pages;
        if (response.page != requestedPage || totalPages < 0
            || (totalPages > 0 && totalPages < requestedPage))
            throw new IOException("Invalid pagination");
        if (totalPages == 0 && !response.data.isEmpty())
            throw new IOException("Nonempty data with zero pages");
    }

    private static void appendMatches(
        AuthorPage page, int threshold, List<String> result)
        throws IOException {
        for (Author author : page.data) {
            if (author == null || author.username == null
                || author.submission_count == null)
                throw new IOException("Missing author fields");
            if (author.submission_count > threshold)
                result.add(author.username);
        }
    }

    public static final class AuthorPage {
        public Integer page;
        public Integer per_page;
        public Integer total;
        public Integer total_pages;
        public List<Author> data;
    }

```

```

    public static final class Author {
        public Integer id;
        public String username;
        public String about;
        public Integer submitted;
        public String updated_at;
        public Integer submission_count;
        public Integer comment_count;
        public String created_at;
    }
}

```

4 Example Calls

The following example calls both solutions. The pangram result is 1010. The API call returns usernames with a submission count greater than 10 and requires internet access.

```

package interview;

import java.util.List;

public class Main {
    public static void main(String[] args) throws Exception {
        List<String> sentences = List.of(
            "abcdefghijklmnopqrstuvwxyz",
            "hello world",
            "the quick brown fox jumps over the lazy dog",
            "abc");

        String result = Pangrams.areTheyPangrams(sentences);
        System.out.println("Pangrams: " + result); // 1010

        ActiveAuthors authors = new ActiveAuthors();
        List<String> usernames = authors.getUsernames(10);
        System.out.println("Active authors: " + usernames);
    }
}

```

5 Interview Feedback

Pangram question. Keep this as a short warm-up to assess basic coding and string handling. Give it limited weight when evaluating experienced developers.

REST API question. Keep this to assess API integration, pagination, filtering, and handling of failures. Evaluate the candidate’s reasoning and tests alongside the output.

5.1 Add a debugging question in HackerRank

Provide an existing implementation with a few deliberate defects and ask the candidate to identify the issues, fix the code, and add tests. This complements the current questions by assessing how a candidate understands and repairs existing code.

5.2 Include code review in the live interview

Ask the candidate to review a short code sample and explain the changes they would recommend. Assess whether they identify correctness issues, missing tests, unclear structure, and weak error handling, and can prioritize their feedback. This evaluates engineering judgment and how clearly they communicate improvements to another developer.

5.3 Include system design in the live interview

Keep system design as a discussion in the live interview, separate from the HackerRank assessment. Use it to evaluate how the candidate clarifies requirements, structures a solution, and explains tradeoffs around scalability, reliability, and data storage. Adjust the depth to the role and experience level.

5.4 Recommendation

Use the pangram and REST API questions to check coding fundamentals and backend integration, and add one debugging question to the HackerRank assessment. Follow with code review and system design discussions in the live interview to assess engineering judgment, architectural reasoning, and communication. Together, these provide broader evidence of how a candidate writes, repairs, reviews, and designs software than the two coding questions alone.